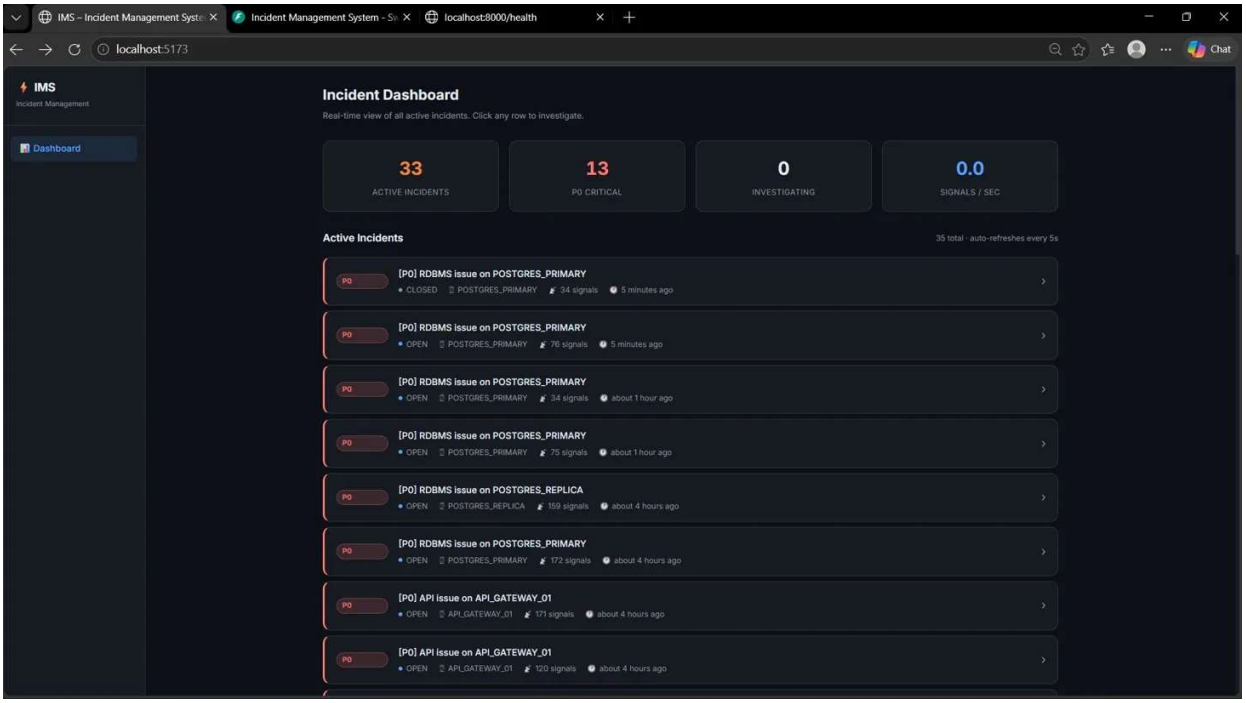
Incident Management System

Product Demonstration -*End-to-End Incident Lifecycle.*

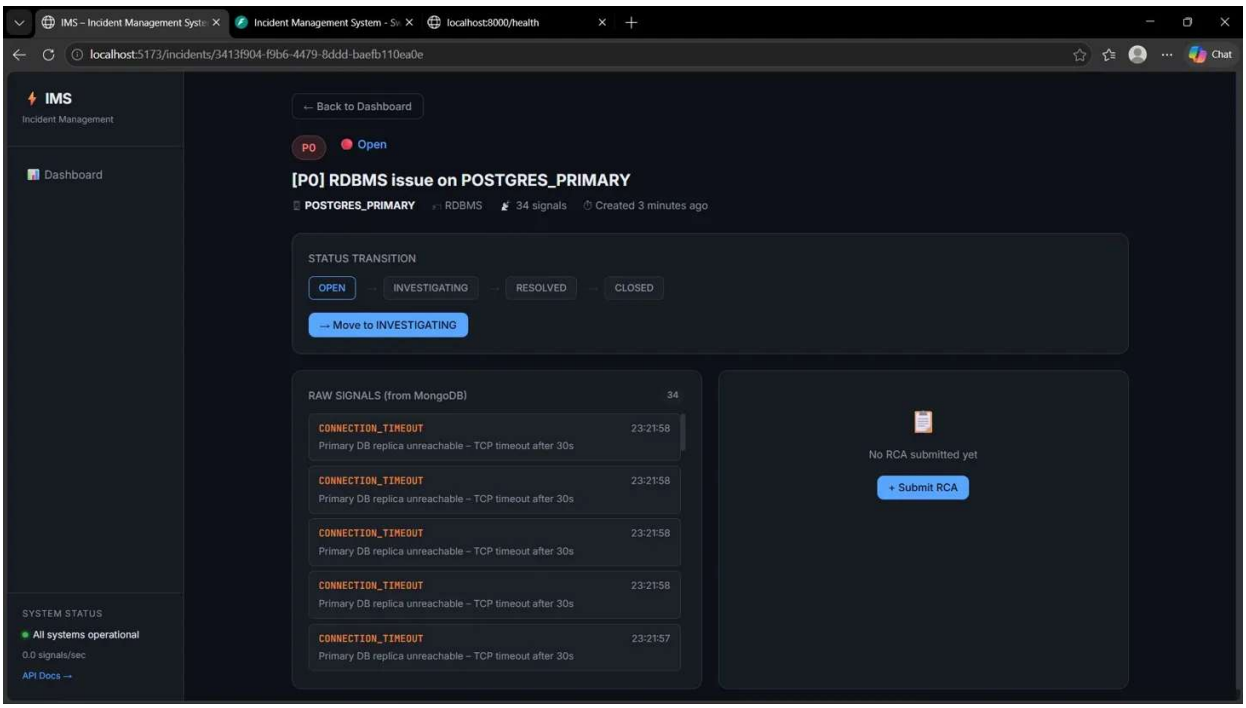
The Incident Management System is a production-grade, full-stack platform built to monitor distributed infrastructure and manage failure workflows end to end. It ingests high-volume monitoring signals from databases, caches, API gateways, and message queues — handling burst traffic using an in-memory ring buffer and async worker pipeline, ensuring the system never crashes under load. A debounce engine groups repeated signals from the same component into a single incident, preventing alert noise. Each incident follows a strict state machine lifecycle from Open to Closed, with a mandatory Root Cause Analysis gate that blocks closure until engineers document the cause, fix, and prevention steps — with Mean Time To Repair calculated automatically. The backend is built with FastAPI and Python, backed by PostgreSQL, MongoDB, and Redis, while the React frontend provides a live incident feed, signal viewer, and RCA form — all auto-refreshing in real time. Below are the screenshots of demonstration.

1- Incident Dashboard (Live Feed)



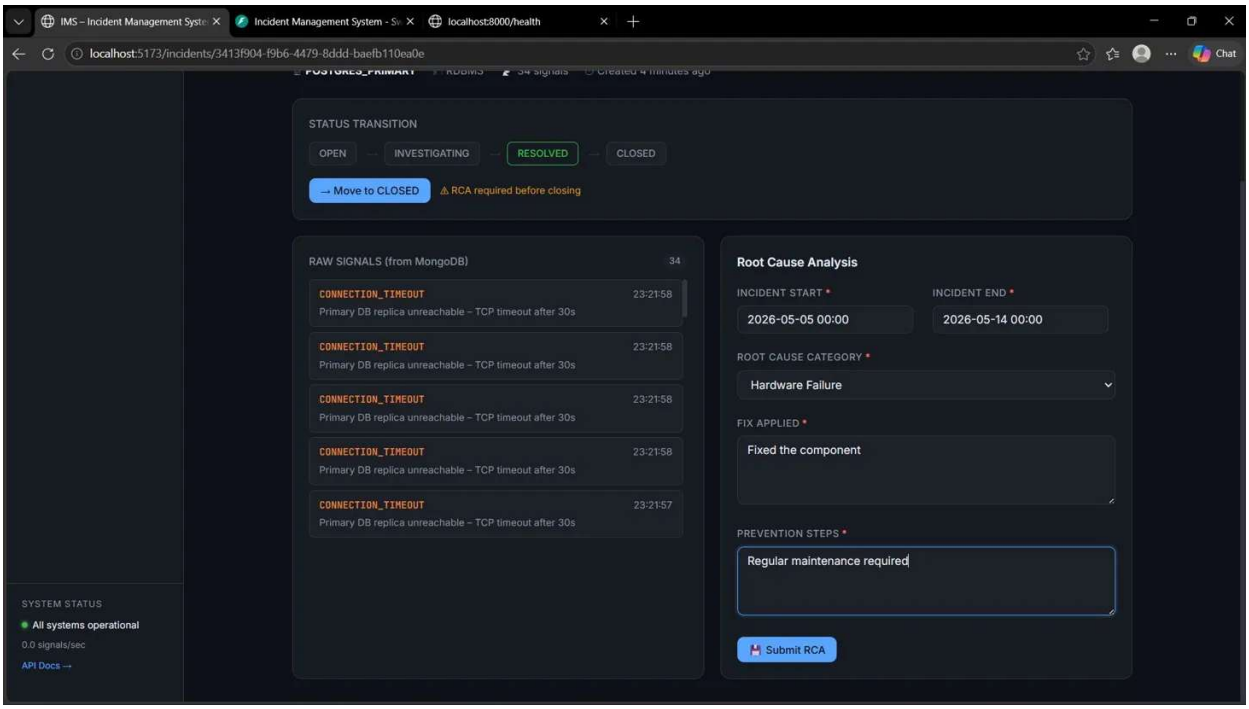
The main command centre of the IMS. Displays real-time metrics showing 33 active incidents, 13 P0 critical alerts, and live signals per second. All incidents are listed in priority order with component name, signal count, status, and age — auto-refreshing every 5 seconds.

2- Incident Detail View (Signal Audit Log):



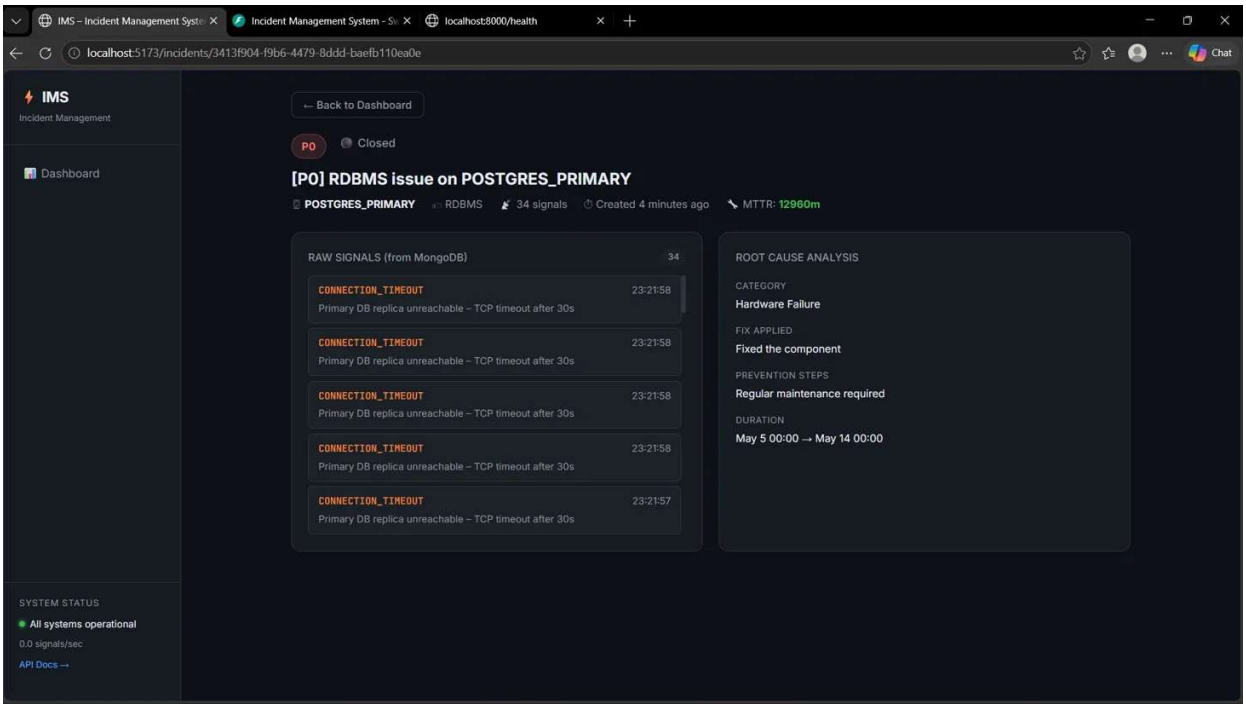
The drill-down page for a single P0 incident on POSTGRES_PRIMARY. Shows the full status transition pipeline (OPEN → INVESTIGATING → RESOLVED → CLOSED), and pulls all 34 raw signals directly from MongoDB in real time, proving the audit log is live and query able.

3- Root Cause Analysis Form:



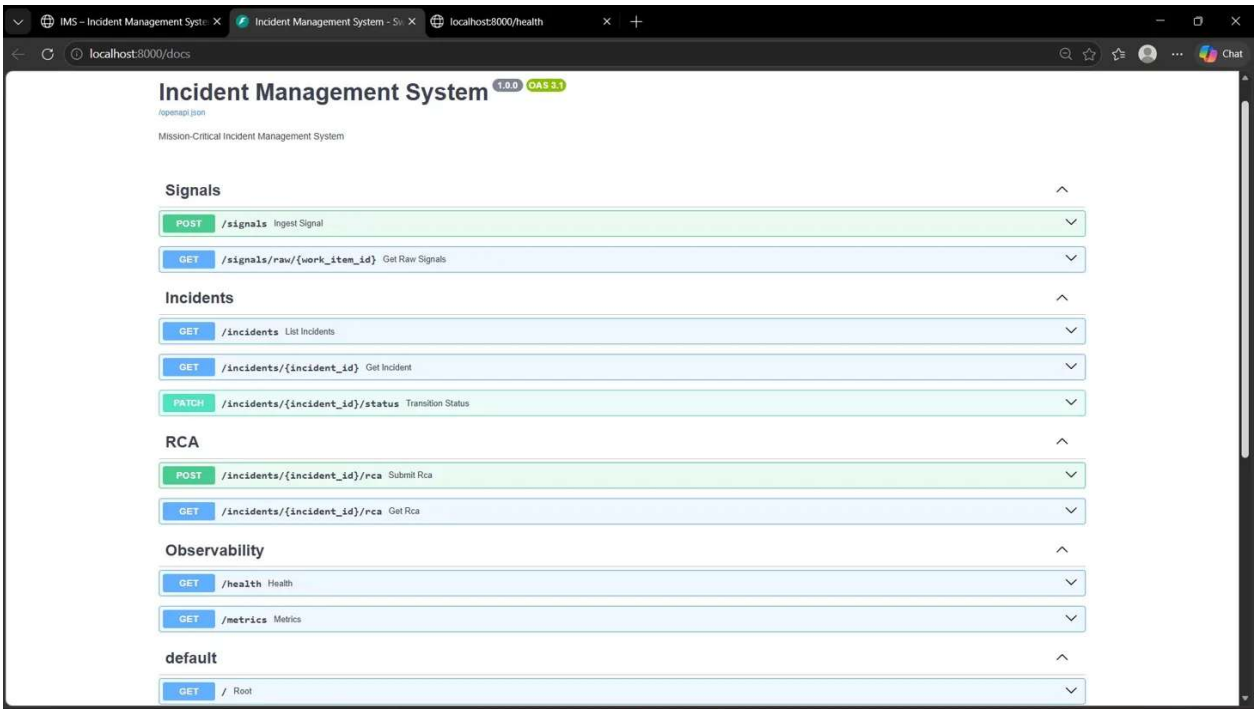
The mandatory RCA form that engineers must complete before an incident can be closed. Captures incident start and end timestamps, root cause category (Hardware Failure), fix applied, and prevention steps — all validated before submission is accepted.

4- Closed Incident with MTTR:



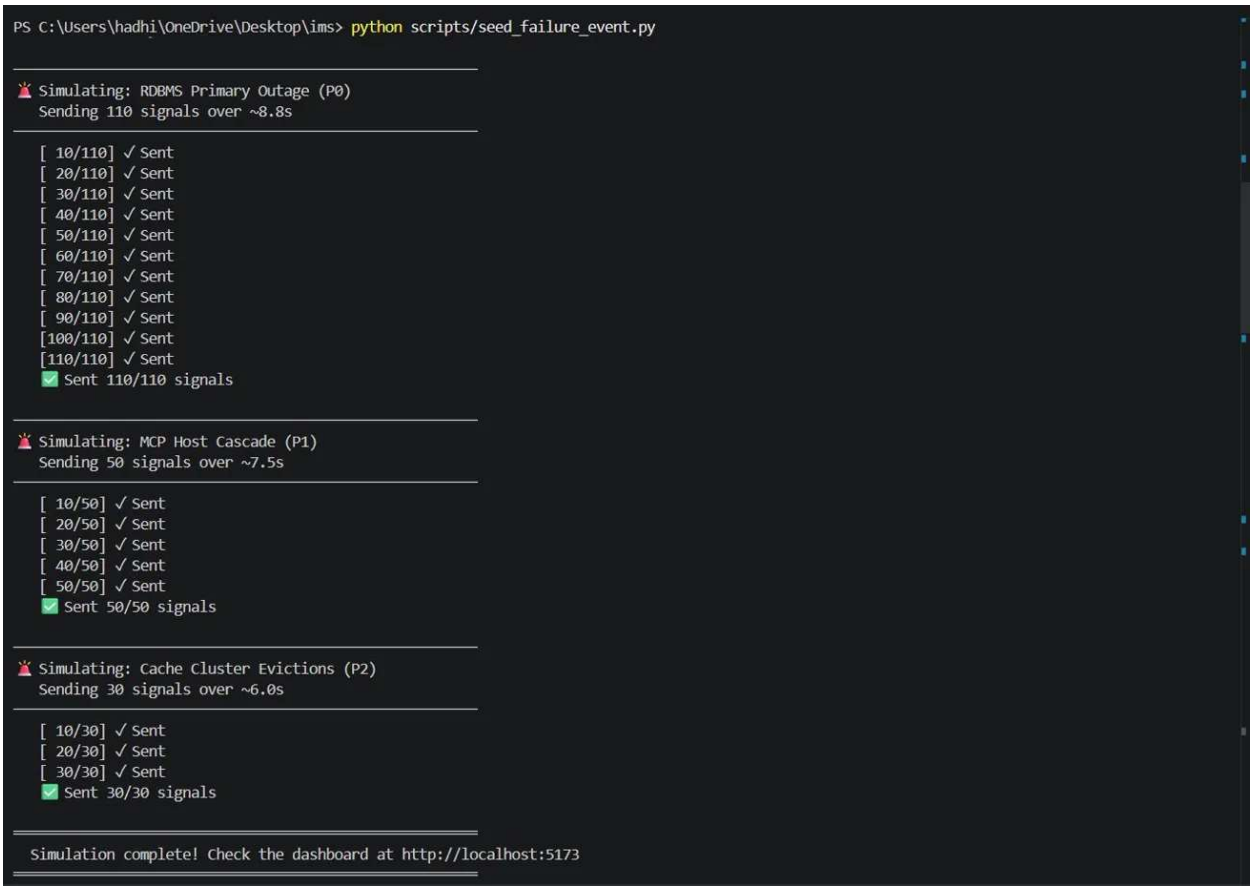
A fully resolved and closed incident showing the completed RCA details alongside the automatically calculated MTTR of 12,960 minutes. Demonstrates the full lifecycle — from open alert to closed incident with documented root cause, fix, and prevention steps.

5- API Documentation:



The interactive API documentation at localhost:8000/docs, showing all 9 endpoints grouped by function: Signals ingestion, Incidents management, RCA submission, and Observability (health + metrics). Every endpoint is live-testable directly from the browser.

6- Failure Simulation Script (Terminal):



The seed_failure_event.py script running in PowerShell, simulating a real cascading infrastructure failure across three severity levels. It sends 110 signals for a P0 RDBMS Primary Outage, 50 signals for a P1 MCP Host Cascade, and 30 signals for a P2 Cache Cluster Eviction — all 190 signals delivered successfully, triggering live incidents on the dashboard.

7- Load Test Results (Terminal):

```
PS C:\Users\hadhi\OneDrive\Desktop\ims> python scripts/load_test.py

Load Test: 200 signals/sec for 20s = 4,000 total
Target: http://localhost:8000

Backend: ok

  [ 20.1s] Sent:  3,040 | OK:  3,040 | 429/err:    0 | Rate:    151/sec

Results:
  Duration:      20.1s
  Total sent:    3,040
  Successful:    3,040
  Rate limited:  0
  Actual rate:   151/sec
```

The load_test.py script demonstrating the system's high-throughput ingestion capability. It sent 3,040 signals at 151 signals/sec over 20 seconds with a 100% success rate — zero rate-limited, zero errors — proving the ring buffer and async worker pipeline handle burst traffic without any data loss or system crash.